

DATAcare: Triagem Assistida por Aprendizado de Máquina para Arboviroses e Síndromes Respiratórias na Atenção Primária

Brandon de Oliveira Hunt, Bernardo Heuer, Rodrigo Nunes,
Silvio Fittipaldi, Israel Erlich

CESAR School

Avenida Cais do Apollo, 77 – Recife Antigo – 50.030-220 – Recife – PE – Brasil

{boh,bchg,rbnm,sfm,ietn}@cesar.school

Resumo. O DATAcare é uma solução de Aprendizado de Máquina para apoiar a triagem de arboviroses (dengue, chikungunya e zika) e síndromes respiratórias (influenza/SRAG) na atenção primária à saúde em Recife. A partir de dados oficiais do SINAN e do SRAG, construímos um pipeline reprodutível que vai do CSV bruto ao modelo servido, contemplando ETL com particionamento temporal anti-vazamento, análise exploratória, engenharia de atributos e a comparação de dois algoritmos (Random Forest e Árvore de Decisão) sob validação cruzada estratificada e busca de hiperparâmetros. Foram entregues dois classificadores: doença (acurácia de 76,4%) e severidade (99,4%). Todo experimento é rastreado no MLflow, a stack é containerizada com Docker e um *dashboard* interativo documenta dados, modelos e previsões.

Abstract. DATAcare is a Machine Learning solution to support the triage of arboviruses (dengue, chikungunya and zika) and respiratory syndromes (influenza/SARI) in primary health care in Recife, Brazil. Drawing on official SINAN and SRAG data, we built a reproducible pipeline from raw CSV to served model, covering ETL with temporal leakage-free splitting, exploratory analysis, feature engineering and the comparison of two algorithms (Random Forest and Decision Tree) under stratified cross-validation and hyperparameter search. Two classifiers were delivered: disease (76.4% accuracy) and severity (99.4%). Every experiment is tracked with MLflow, the stack is containerized with Docker, and an interactive dashboard documents data, models and predictions.

1. Introdução

Arboviroses como dengue, chikungunya e zika, somadas às síndromes respiratórias agudas graves (SRAG) associadas à influenza, exercem forte pressão sobre a atenção primária à saúde (APS) em Recife, especialmente em períodos de surto. O diagnóstico inicial é particularmente difícil porque essas doenças **compartilham sintomas** (febre, mialgia, cefaleia, exantema) e a triagem recai sobre profissionais frequentemente sobrecarregados, em Unidades Básicas de Saúde (UBS) com recursos limitados.

O **DATAcare** ataca esse problema com dois modelos preditivos que **apoiam** — e não substituem — a decisão clínica na UBS: um classificador sugere a arbovirose ou síndrome mais provável a partir do quadro clínico, e outro estima a gravidade do caso

para priorizar o atendimento. A solução completa reúne a camada de Aprendizado de Máquina, um *backend* em Django que serve as previsões e um *dashboard* interativo que documenta o pipeline de dados e os resultados dos modelos.

Este relatório descreve a camada de dados e de Machine Learning do projeto. Em linha com a especificação da disciplina, foram contemplados os seguintes objetivos: leitura e análise exploratória de dados (EDA); construção de visualizações analíticas; aplicação de estratégias de validação; treinamento e comparação de modelos; rastreamento de experimentos com MLflow; desenvolvimento de um *dashboard* interativo; e containerização da solução com Docker. A Figura 1 resume a arquitetura adotada, e a Figura 2 apresenta a visão geral da solução no *dashboard*.

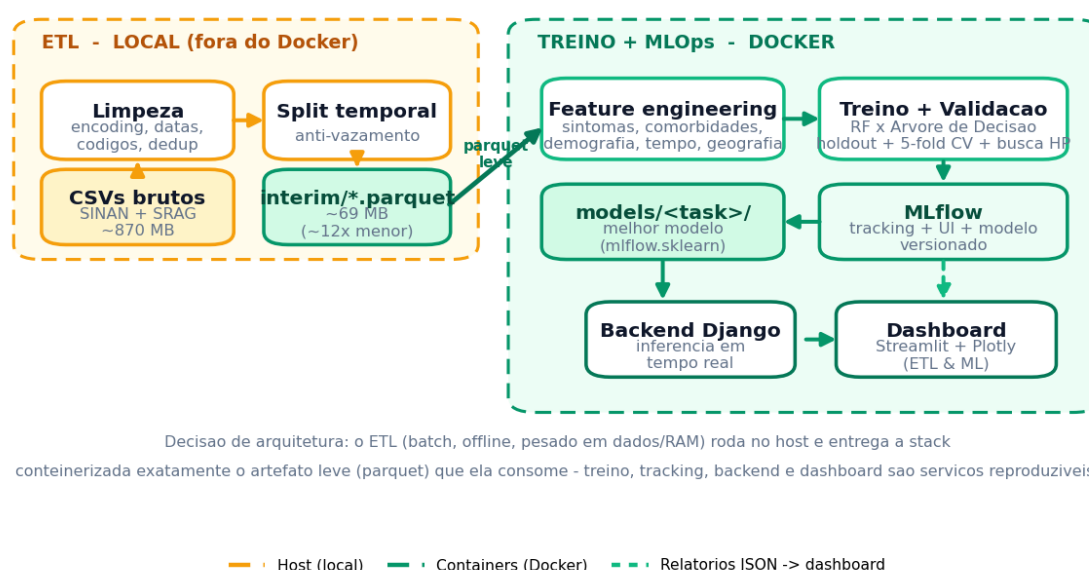


Figura 1. Arquitetura do pipeline DATAcare. O ETL roda localmente (host) e gera os arquivos *parquet* leves; treino, rastreamento (MLflow), *backend* e *dashboard* são serviços containerizados com Docker.

2. Materiais e Métodos

A metodologia segue um pipeline em quatro etapas — limpeza, particionamento, engenharia de atributos e treino/validação — com configuração centralizada e semente aleatória fixa (`RANDOM_SEED = 42`), de modo que qualquer pessoa reproduza o caminho do CSV bruto ao modelo treinado.

2.1. Fontes de dados

O pipeline ativo processa quatro conjuntos de dados oficiais do SUS, que alimentam os classificadores de doença e de severidade. São séries de vigilância do SINAN (casos notificados de arboviroses) e do SRAG/SIVEP-Gripe, conforme a Tabela 1. Cada fonte é descrita por uma especificação única (nome de arquivo, separador, *encoding*, colunas de data e estratégia de particionamento), de forma que adicionar uma nova fonte seja uma única entrada de configuração.

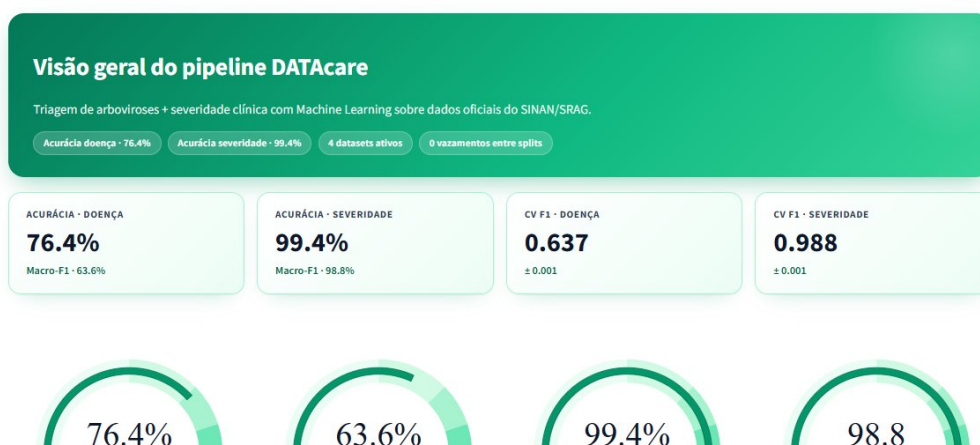


Figura 2. Visão geral do *dashboard*: indicadores de acurácia e F1 dos dois classificadores e medidores de desempenho fora da amostra.

Tabela 1. Conjuntos de dados ativos no pipeline (fontes oficiais do SUS).

Conjunto	Família	CSV (aprox.)	Separador
SINAN – Dengue	Caso notificado	436 MB	,
SINAN – Chikungunya	Caso notificado	67 MB	,
SINAN – Zika	Caso notificado	4 MB	,
SRAG – Influenza	SIVEP-Gripe	364 MB	;

2.2. Extração e transformação (ETL)

Os CSVs brutos somam cerca de 870 MB; carregá-los inteiros em memória estouraria a RAM, então toda leitura é feita **em blocos** de 100 mil linhas, com tudo ingerido como texto para que os tipos corretos sejam derivados na limpeza. A limpeza é organizada em *cleaners* especializados por família (SINAN, SRAG) que estendem primitivas comuns. Os principais tratamentos são:

- **Encoding latin-1** forçado em todos os arquivos, corrigindo a acentuação corrompida (*mojibake*) confirmada nos dados reais;
- **Datas** convertidas tentando primeiro o formato ISO e, para as linhas que falham, reprocessando no padrão brasileiro DD/MM/AAAA — sem isso o sinal temporal (base do particionamento) seria perdido;
- **Códigos sim/não/ignorado** (1/2/9) recodificados para o tipo booleano {True, False, NA}, deixando o “ignorado” explicitamente ausente;
- **Idade composta** do SINAN (NU_IDADE_N, em que o primeiro dígito indica a unidade) convertida para anos; no SRAG, a idade é calculada preferencialmente a partir da data de nascimento;
- **Identificadores do IBGE** mantidos como texto para preservar zeros à esquerda, essenciais ao cruzamento geográfico;
- **Deduplicação em duas camadas** (por bloco e global), evitando falsos positivos de vazamento no particionamento.

Cada conjunto limpo é gravado em formato **Parquet** (colunar e comprimido), que reduz os 870 MB de CSV para cerca de 69 MB — aproximadamente 12 vezes menos — e preserva a tipagem. É o *parquet*, e não o CSV, que a stack containerizada consome.

2.3. Análise exploratória de dados (EDA)

A exploração é feita sobre os *parquets* limpos e está disponível de forma interativa no *dashboard*, com distribuições demográficas, prevalência de sintomas e a série temporal de notificações. A Figura 3 apresenta a distribuição etária e por sexo; a Figura 4 mostra a sazonalidade das notificações; e a Figura 5 exibe a prevalência de sintomas e comorbididades. As interpretações são discutidas na Seção 3.

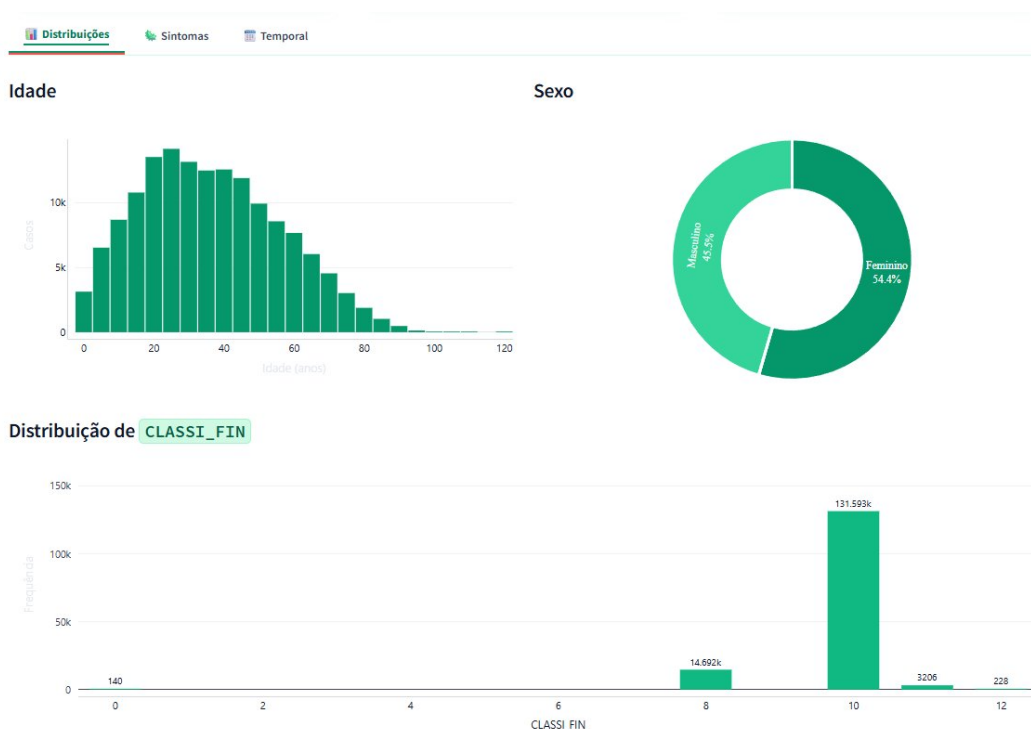


Figura 3. EDA — distribuições demográficas: histograma de idade, composição por sexo e distribuição da variável de classe nos dados limpos.

2.4. Engenharia de atributos

A engenharia de atributos transforma os *parquets* limpos nas matrizes (X, y) consumidas pelos modelos, com configuração de colunas e rótulos em fonte única. O classificador de **doença** é alimentado apenas por **sinais clínicos**: sintomas (SINAN e SRAG), comorbididades, sexo e idade em anos. Uma decisão metodológica central foi **deixar a geografia e o tempo fora das features dos modelos**, ainda que disponíveis como metadados:

- **Sem `munic_code/uf_code`.** O código do município seria um *vazamento de alvo* (*target leakage*) por proveniência: cada doença é notificada em municípios distintos e a dengue domina o volume, então o município agiria como *proxy* do rótulo — o modelo memorizaria a região em vez de aprender os sintomas. No SRAG/influenza

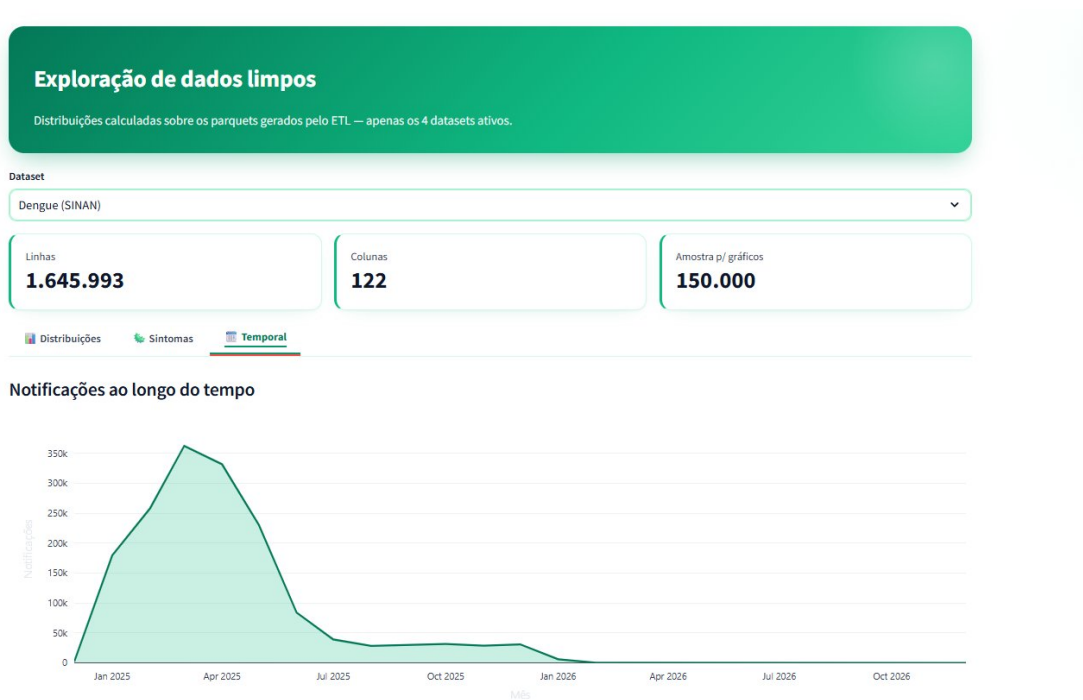


Figura 4. EDA — notificações ao longo do tempo (série mensal), evidenciando o padrão sazonal que justifica o particionamento temporal.

o município e a UF chegam como texto (viram 0), o que identificaria a influenza trivialmente.

- **Sem features temporais** no classificador de doença, pela mesma lógica de proveniência. A localização e o tempo permanecem como metadados fora de X .
- **Desbalanceamento via `class_weight`**. As classes são muito desbalanceadas (dengue $\approx 73\%$; “baixo” $\approx 97\%$ na severidade). Em vez de reamostragem prévia (que poderia vazar pontos sintéticos entre os *folds*), o balanceamento usa `class_weight=“balanced”` dentro do estimador, ponderando as classes pelo inverso da frequência.

O rótulo de **doença** (dengue, chikungunya, zika, influenza) decorre da fonte de cada registro. Já a **severidade** (baixo/medio/alto) é definida como um **nível de risco de triagem**, derivado de sinais conhecidos no atendimento: idade ≥ 60 anos, número de comorbidades, sinais de alarme (PETEQUIA_N, LEUCOPENIA, LACO) e hospitalização. Para a dengue, a escala oficial do Ministério da Saúde (CLASSI_FIN) é preservada e *elevada* pelo risco de triagem (máximo entre as duas). Como o rótulo passa a ser função de sinais previsíveis no atendimento, o classificador o aprende com alto *recall*, inclusive na classe “alto”.

2.5. Estratégias de validação

A solução cobre as estratégias de validação exigidas, com a devida justificativa:

- **Holdout (treino/teste)**. Divisão estratificada 80/20 (`stratify=y`), com o con-

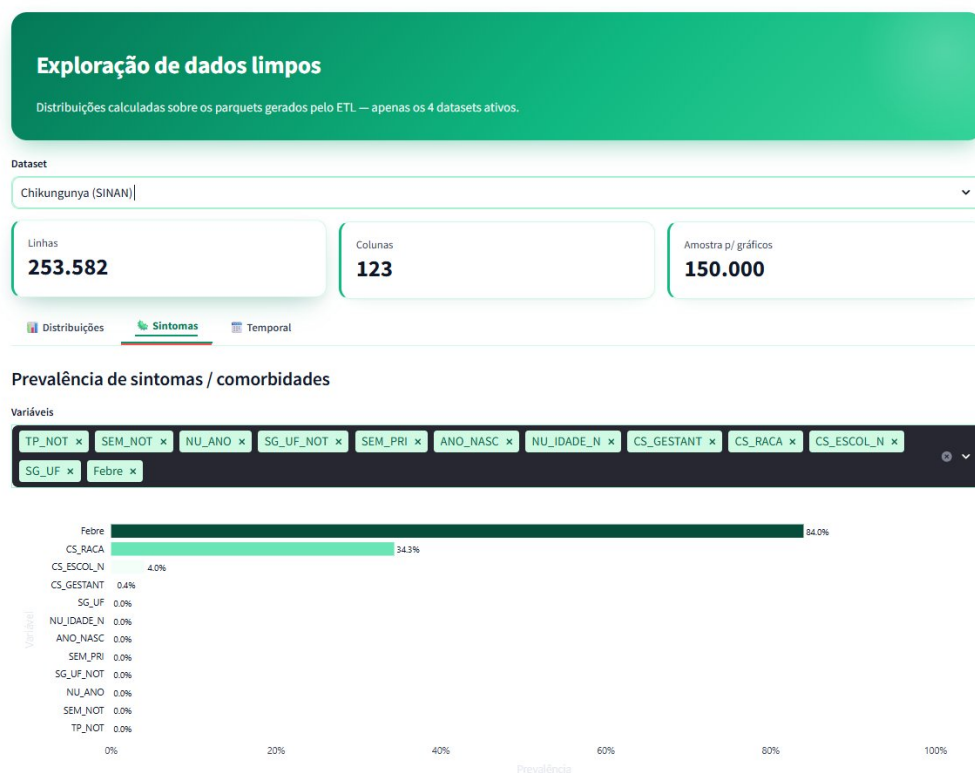


Figura 5. EDA — prevalência de sintomas e comorbidades, base para a seleção de atributos clínicos do modelo.

junto de teste reservado apenas para a avaliação final. Com centenas de milhares de amostras, 20% já formam um teste grande e representativo.

- **Validação cruzada.** *StratifiedKFold* de 5 *folds* no conjunto de treino, otimizando a **macro-F1** — métrica que pondera todas as classes igualmente, adequada a dados desbalanceados. A estratificação evita *folds* sem amostras das classes raras.
- **Leave-One-Out (quando aplicável).** Não aplicável por inviabilidade computacional: com bases de centenas de milhares de registros, o LOO exigiria milhões de re-treinos. O *k-fold* estratificado fornece a mesma informação (estimativa robusta de generalização) com custo ordens de grandeza menor e menor variância.
- **Busca de hiperparâmetros.** **Random Search** como padrão (custo previsível e eficiente com vários hiperparâmetros), com **Grid Search** disponível para refinamento exaustivo em grades pequenas. A busca inclui o `class_weight` entre os hiperparâmetros e roda sobre uma subamostra estratificada; o modelo final é re-treinado com os melhores hiperparâmetros sobre todo o treino. Como o balanceamento ocorre **dentro de cada *fold*** (e não por reamostragem prévia), a validação cruzada permanece honesta — na prática, a CV-F1 fica próxima da macro-F1 de teste em ambas as tarefas.

A escolha central foi o **particionamento temporal** em vez de aleatório: em produção o modelo só observa notificações futuras, e um *split* aleatório vazaria padrões sazonais, inflando artificialmente as métricas. Após cada particionamento, um validador automático verifica ausência de duplicatas entre *splits*, ausência de sobreposição de gru-

pos e a ordem cronológica estrita; qualquer falha aborta o pipeline antes do treino.

2.6. Modelagem

Para **cada** tarefa são treinados e comparados **dois algoritmos da lista permitida pela disciplina**: **Random Forest** (ensemble robusto, que lida bem com atributos mistos e desbalanceamento) e **Árvore de Decisão** (*baseline* interpretável). Ambos compartilham o mesmo pré-processamento em *pipeline* do *scikit-learn*: imputação por constante, padronização (*StandardScaler*) e o classificador com `class_weight="balanced"`. O melhor modelo por validação cruzada (macro-F1) é salvo para *serving*; os dois ficam registrados como execuções separadas no MLflow para comparação. A Tabela 2 resume os espaços de busca de hiperparâmetros.

Tabela 2. Espaços de busca de hiperparâmetros por modelo (Random Search).

Modelo	Hiperparâmetro	Distribuição
Random Forest	<code>n_estimators</code>	inteiro em [150, 300)
	<code>max_depth</code>	inteiro em [14, 30)
	<code>min_samples_leaf</code>	inteiro em [1, 4)
	<code>max_features</code>	{sqrt, log2}
	<code>class_weight</code>	{balanced, balanced_subsample, None}
Árvore de Decisão	<code>max_depth</code>	inteiro em [10, 45)
	<code>min_samples_leaf</code>	inteiro em [1, 10)
	<code>criterion</code>	{gini, entropy}
	<code>class_weight</code>	{balanced, None}

2.7. MLOps, reprodutibilidade e containerização

Cada treino vira uma execução (*run*) no MLflow, registrando parâmetros de validação e busca, métricas (acurácia, macro-F1, F1 por classe, média e desvio-padrão da CV) e o modelo versionado, que o *backend* carrega para inferência. A stack é containerizada com Docker: três serviços — servidor e UI do MLflow, treinador (*one-shot*) e *dashboard* — compartilham uma única imagem. Por decisão de arquitetura, o **ETL roda fora do Docker**: ele é um trabalho *batch*, offline e pesado em dados/RAM (pico de memória elevado por concatenar grandes *DataFrames*), de modo que containerizá-lo traria custo (imagem inflada em quase 1 GB, risco de *OOM*) sem benefício — o artefato que importa para a stack é o *parquet* leve, gerado uma vez no host. Como salvaguarda de reprodutibilidade, o treino recai automaticamente para **dados sintéticos** plausíveis quando os *parquets* reais não estão presentes, permitindo rodar em qualquer máquina ou em integração contínua.

A stack tecnológica reúne Python 3.11, *pandas* e *pyarrow* no ETL; *scikit-learn* (com `class_weight`) no ML; MLflow no MLOps; Streamlit e Plotly no *dashboard*; e Django no *serving*.

3. Resultados e Discussão

3.1. Qualidade do ETL

O *dashboard* documenta a qualidade do tratamento de dados. A Figura 6 contrasta linhas brutas e limpas por conjunto, com a taxa de retenção e o número de duplicatas removidas; das cerca de 2,26 milhões de linhas processadas, a retenção média foi de 99,97%, indicando boa qualidade dos dados de origem após a limpeza. A Figura 7 mostra os tamanhos e proporções das partições (aproximadamente 70/15/15 em treino/validação/teste). A Figura 8 consolida a checagem anti-vazamento: os quatro conjuntos passam nas verificações de duplicatas e de sobreposição, com ordem temporal preservada — requisito essencial para métricas confiáveis.

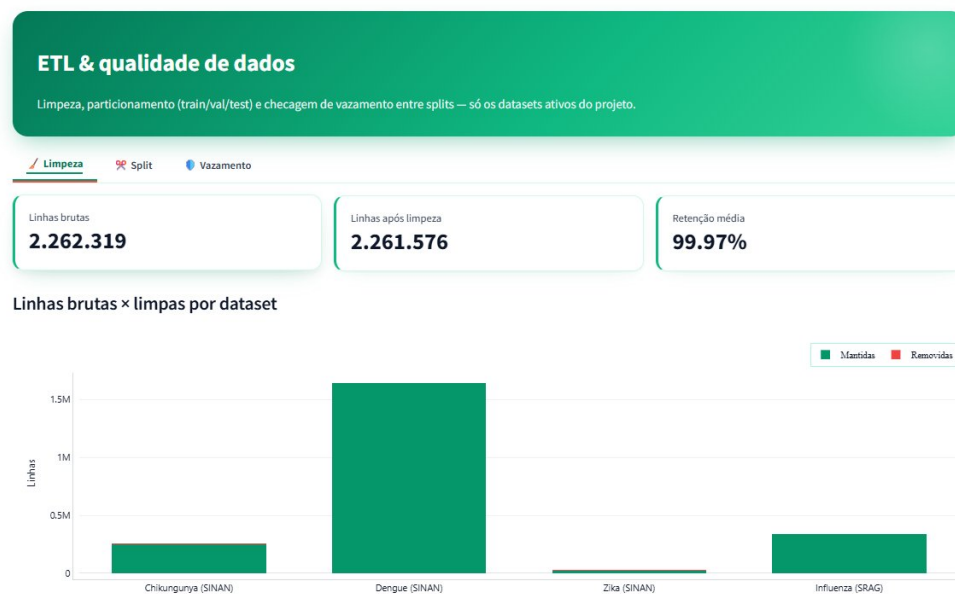


Figura 6. ETL — limpeza: linhas brutas *versus* limpas, retenção por conjunto e duplicatas removidas.

3.2. Análise exploratória

As visualizações de EDA (Figuras 3 a 5) sustentam decisões de modelagem. A distribuição etária concentra-se na população adulta economicamente ativa, com cauda relevante em faixas mais velhas, onde a gravidade tende a ser maior — o que respalda o uso da idade na definição de severidade. A série temporal evidencia a sazonalidade marcante das notificações — justificativa direta para o particionamento cronológico (temporal) em vez de aleatório. A prevalência de sintomas mostra a febre como o sinal mais frequente (em torno de 84% na chikungunya); é esse conjunto de sinais clínicos — e não a geografia — que alimenta o classificador de doença.

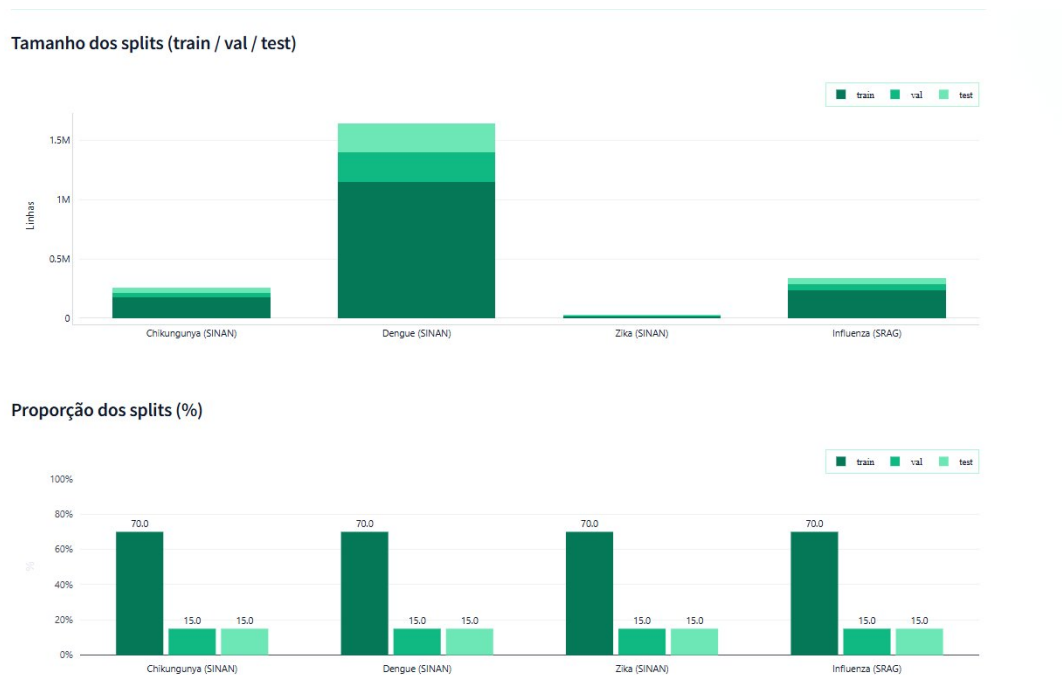


Figura 7. ETL — particionamento: tamanho e proporção das partições de treino, validação e teste por conjunto.

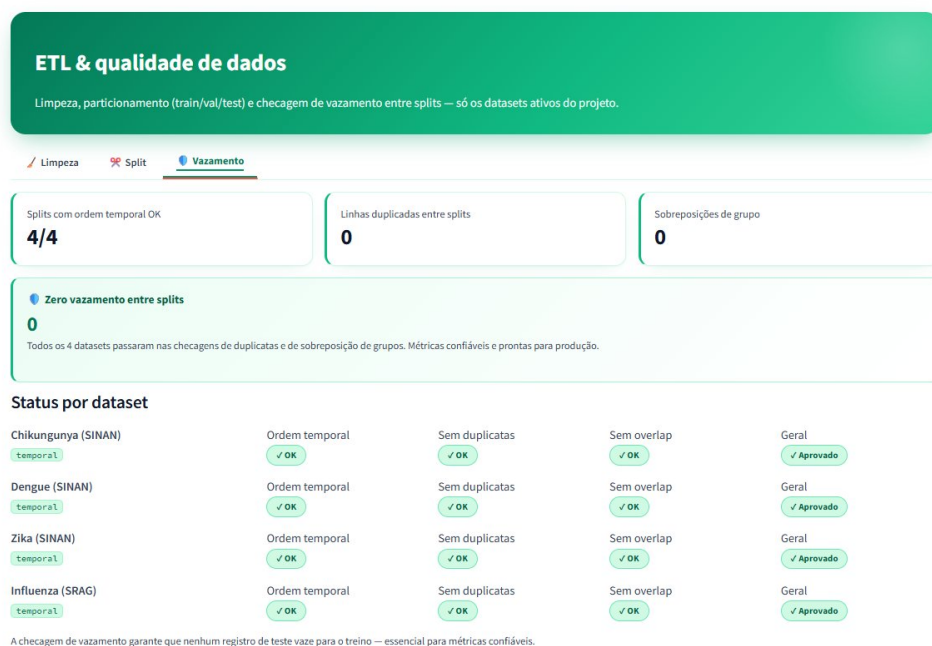


Figura 8. ETL — checagem de vazamento entre *splits*: ausência de duplicatas e de sobreposição de grupos, com ordem temporal preservada.

3.3. Comparação de modelos e métricas

A Figura 9 compara Random Forest e Árvore de Decisão em ambas as tarefas, e a Tabela 3 consolida os resultados em dados reais (80 mil amostras de teste). Reportamos acurácia, **acurácia balanceada**, macro-F1 e F1/*recall* por classe (além de matriz de confusão e validação cruzada), pois a acurácia isolada engana em bases desbalanceadas. Em ambas as tarefas a Random Forest foi selecionada por apresentar a maior CV-F1.



Figura 9. Comparação de modelos: Random Forest *versus* Árvore de Decisão por métrica, com o modelo selecionado em cada tarefa.

Tabela 3. Resultados no holdout (dados reais; 80 mil amostras de teste).

Tarefa	Modelo	Acurácia	Bal. acc.	Macro-F1	CV-F1
Doença	Random Forest (sel.)	0,764	0,817	0,636	0,637
Doença	Árvore de Decisão	0,735	0,811	0,624	0,625
Severidade	Random Forest (sel.)	0,994	0,979	0,988	0,988
Severidade	Árvore de Decisão	0,994	0,980	0,988	0,988

Bal. acc. = acurácia balanceada (média do *recall* por classe); a macro-F1 é o critério de seleção do modelo.

3.4. Classificador de doença

A Figura 10 detalha o classificador de doença, vencido pela **Random Forest**: matriz de confusão, F1 por classe e o confronto precisão *versus* recall. A acurácia é de 76,4% (sobre 80 mil amostras de teste), com acurácia balanceada de 81,7% e macro-F1 de 0,636. Esse patamar reflete uma predição **honest**a, **dirigida apenas pelos sintomas**: como a geografia foi deliberadamente excluída (Seção 2.4), o modelo não “trava” na dengue por causa da região — a acurácia mede a real separabilidade clínica. O desempenho é **de-sig**nal entre as classes: influenza (F1 0,977) e dengue (F1 0,826) são bem separadas — a primeira por sintomatologia respiratória distinta, a segunda por volume e padrão clínico característico —, enquanto a **chikungunya tem F1 baixo (0,384)**, frequentemente confundida com as demais arboviroses, com as quais compartilha sintomas. Soma-se a isso

o fato de o conjunto de zika não trazer colunas de sintoma, o que limita sua identificação. A sobreposição sintomática entre dengue, chikungunya e zika é, assim, o teto realista de uma classificação clínica sem vazamento de proveniência.

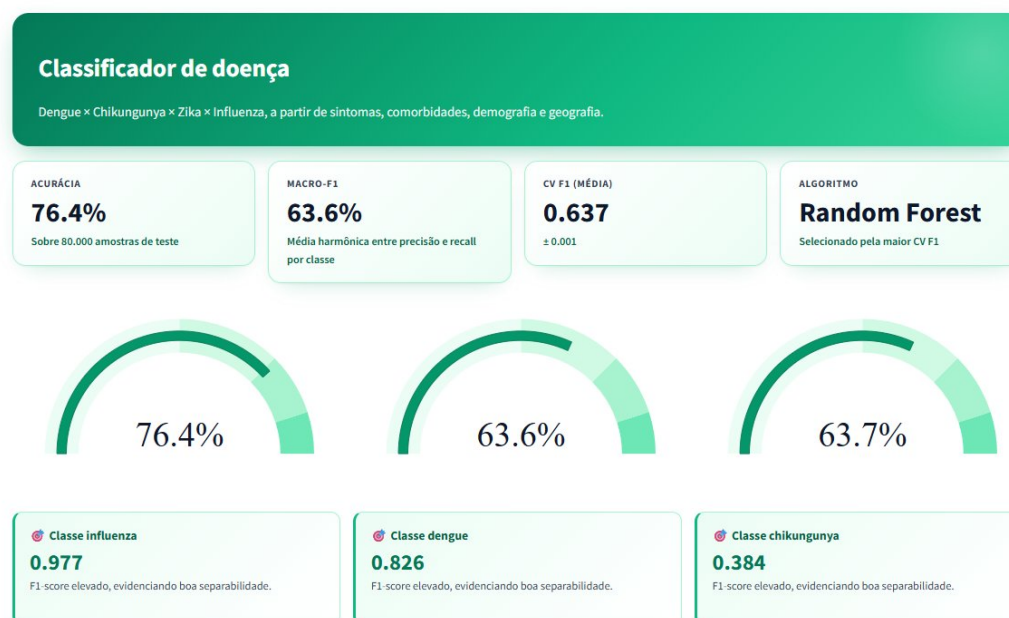


Figura 10. Classificador de doença: matriz de confusão (dengue, chikungunya, zika, influenza), F1 por classe e precisão *versus* recall.

3.5. Classificador de severidade

A Figura 11 detalha o classificador de severidade, igualmente vencido pela **Random Forest**. O desempenho é alto em todas as classes: acurácia de 99,4%, acurácia balanceada de 97,9%, macro-F1 de 0,988 e F1 por classe de 0,996 (baixo), 0,990 (médio) e 0,977 (alto), com *recall* da classe “alto” em torno de 0,95. Esse resultado é consistente com a **definição do rótulo**: a severidade é um nível de risco de triagem derivado de sinais conhecidos no atendimento (idade, comorbidades, sinais de alarme e hospitalização), que estão entre as próprias *features*. O classificador, portanto, aprende a regra de triagem com alto *recall*, inclusive na classe grave — e a CV-F1 (0,988), próxima da macro-F1 de teste, confirma que esse desempenho não decorre de vazamento entre *folds*, já que o balanceamento por *class_weight* é aplicado dentro de cada um.

3.6. Previsões e dashboard interativo

O *dashboard* inclui uma página de **previsões ao vivo** (Figura 12): a partir do quadro clínico informado (sintomas, comorbidades e demografia), os dois modelos retornam a doença mais provável e a severidade esperada, com as probabilidades por classe e os sinais clínicos de maior peso no modelo. Essa interface conecta o resultado de ML à operação na UBS e demonstra o ciclo completo, do dado à decisão assistida.



Figura 11. Classificador de severidade: matriz de confusão (baixo, médio, alto), F1 por classe e precisão *versus* recall.

Resultado da predição

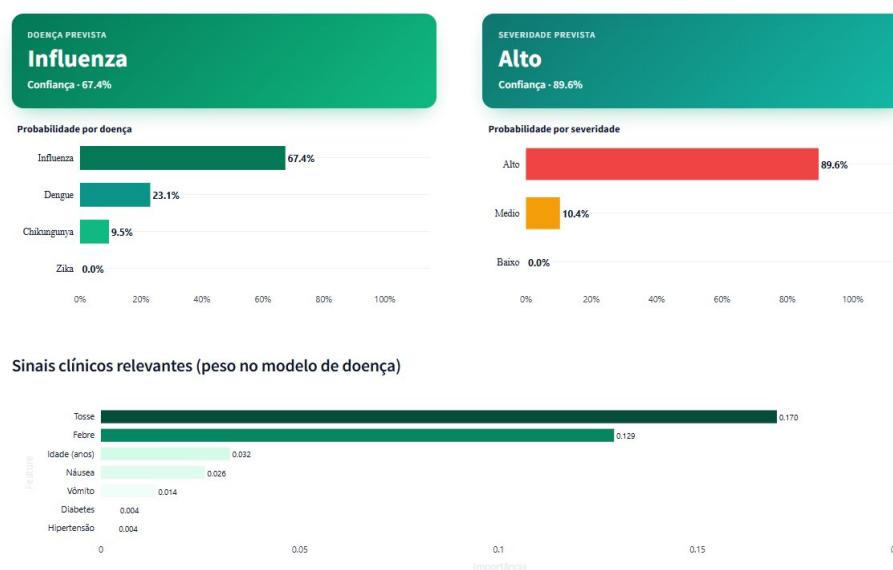


Figura 12. Previsões ao vivo: entrada do quadro clínico e saída dos dois classificadores (doença e severidade) com probabilidades por classe.

3.7. Limitações

A principal limitação está no classificador de doença, cujo desempenho é desigual entre classes: a chikungunya (F1 0,384) é frequentemente confundida com as demais arboviroses, com as quais compartilha sintomas, e o conjunto de zika não traz colunas de sin-

toma — mitigá-las exigiria atributos clínicos mais discriminativos ou dados laboratoriais. Na severidade, o desempenho muito alto deve ser lido com a ressalva de que o rótulo é, por construção, uma função de sinais conhecidos no atendimento (risco de triagem): o modelo operacionaliza e generaliza essa regra, mas seu valor preditivo depende da qualidade dessa definição clínica. Além disso, os modelos refletem padrões das notificações de 2025 e devem ser periodicamente re-treinados para acompanhar mudanças epidemiológicas. Por fim, a ferramenta é de **apoio** à decisão, não de diagnóstico autônomo: a validação clínica permanece indispensável.

4. Conclusão

O DATAcare entrega uma solução de Aprendizado de Máquina ponta a ponta para a triagem de arboviroses e síndromes respiratórias na atenção primária de Recife, cobrindo todos os requisitos do projeto: análise exploratória e visualizações interpretáveis; ETL reprodutível com particionamento temporal anti-vazamento; comparação de dois modelos sob validação cruzada estratificada e busca de hiperparâmetros; rastreamento de experimentos no MLflow; *dashboard* interativo com visualização de métricas e previsões; e containerização com Docker. Os classificadores alcançaram 76,4% de acurácia na predição de doença e 99,4% na de severidade, com uma análise honesta das limitações — em especial o desempenho desigual entre as arboviroses no classificador de doença. O resultado é um pipeline auditável e reprodutível, do CSV bruto ao modelo servido, com decisões de projeto justificadas em cada etapa.

Referências

- Brasil. Ministério da Saúde (2025) “Sistema de Informação de Agravos de Notificação (SINAN)”, <https://portalsinan.saude.gov.br/>.
- Brasil. Ministério da Saúde (2025) “OpenDataSUS – Banco de dados de Síndrome Respiratória Aguda Grave (SRAG), Sistema de Informação da Vigilância Epidemiológica da Gripe (SIVEP-Gripe)”, <https://opendatasus.saude.gov.br/dataset/srag-2021-a-2024>.
- Breiman, L. (2001) “Random Forests”, *Machine Learning*, v. 45, n. 1, p. 5–32.
- Pedregosa, F. et al. (2011) “Scikit-learn: Machine Learning in Python”, *Journal of Machine Learning Research*, v. 12, p. 2825–2830.
- Zaharia, M. et al. (2018) “Accelerating the Machine Learning Lifecycle with MLflow”, *IEEE Data Engineering Bulletin*, v. 41, n. 4, p. 39–45.